

AI Logs – Sprint 1

1. Fixing cross-platform setup issues

Purpose of AI Use: Fixing cross-platform setup problem: backend dependencies were failing to install on my laptop, and our `setup.sh` / `run.sh` scripts needed to work for everyone no matter their OS.

Chat Link or Prompt/Response:

Prompt: My laptop is Windows on ARM64, and some backend dependencies are failing to install . Help me find ARM64-compatible alternatives for each one without changing the app, and fix our `setup.sh`/`run.sh` scripts so they still work cross-platform for teammates (on Windows, Mac,...). here are my current `run.sh` and `setup.sh` files: [pasted current files]

Response: I've fixed both scripts. I still need your `requirements.txt` to pick the ARM64 alternatives, because you didn't include it.

The scripts. I tested both on a small stand-in project on Linux. `setup.sh` installed everything, `run.sh` started both servers, Ctrl+C stopped both, and when one server crashed the other stopped too. I haven't run them on Windows or macOS yet. The changes:

- **Works on Windows now.** The scripts find the right Python whether it's called `python3`, `python` or `py`. They also handle the different folder layout Windows uses inside `venv`.
- **Shuts down properly.** Before, Ctrl+C only stopped the parent processes, so helper processes could keep the ports busy. Now everything they started is stopped, and if either server crashes the other stops too.
- **Clearer errors.** Setup warns early if Python or Node is missing. If an install fails on your ARM64 laptop, it tells you why.
- **New `.gitattributes` file.** It stops Git from converting the scripts to Windows line endings, which breaks them in Git Bash (`$'\r': command not found`). If anyone already has a broken checkout, they should re-clone or run `git add --renormalize .` once.
- On Windows, run them from Git Bash with `bash setup.sh` and then `bash run.sh`.

The packages. I checked PyPI for common backend packages that have ARM64 builds for Windows (using Python 3.12):

Package	Windows ARM64	Alternative
---------	------------------	-------------

psycpg2-binary, psycpg[binary], asynpg	 none	pg8000, which is pure Python and installs anywhere
httptools (comes with uvicorn[standard])	 none	Skip it on ARM64; uvicorn falls back to its built-in HTTP parser automatically
pydantic-core, watchfiles	 available	No change needed

Your app connects to Supabase, so the Postgres driver is almost certainly one of the failures. The fix keeps the regular driver for your teammates by adding two lines to `requirements.txt`:

```
psycpg2-binary; platform_machine != "ARM64"
pg8000; platform_machine == "ARM64"
```

Whether the app works with that as-is depends on how it builds its database connection. If it uses SQLAlchemy, I'd add a small change so the driver is picked automatically.

[attached ai coded-files for setup.sh, run.sh and .gitattributes]

AI-Suggested Content:

- ARM64-compatible alternatives for the backend dependencies that failed to install.
- A rewritten setup.sh
- A rewritten run.sh
- A .gitattributes file

Validation: Installed the new requirements on my laptop to confirm they built without errors. Compared the AI scripts line by line against our repo and ran them to confirm setup and startup worked. Found that the AI's setup.sh copies a backend/.env.example file that doesn't exist in our repo, which would make setup fail.

Decision: Modified and partially accepted.

Reflection: The AI was useful for finding which dependencies needed native builds on Windows and for pointing out the venv layout and Python command differences between operating systems. Its script rewrite was more complex than our team needed and assumed files we don't have, so I took only the fixes that solved our actual problem and kept the scripts easy for everyone to read.

Responsible Person: Diego Chidiac

2. Tasks 3.1: Set up file storage and define accepted formats and size limits (e.g. PDF, DOCX) AND Task 3.2: Implement resume API endpoints (upload, list, download, delete) linked to the user account

Purpose of AI Use: Initial backend for the resume upload feature (database model, request/response schemas, and API endpoints).

Chat Link or Prompt/Response:

Prompt: Help me build the initial backend: a Resume model tied to a user via foreign key, a POST /resume/upload endpoint that accepts a real PDF file, validates its type and size, stores it on disk under a unique filename, and saves a record in the database, plus GET and DELETE endpoints so a user can list and remove their own resumes. I should have 3 new resume.py files (just different folders)

Response: The resume backend is built, and I ran it end to end against a real Postgres database. Everything works: uploading, listing, deleting, all the ways an upload can be rejected, and the check that users can only see and delete their own resumes. Your project already had placeholder resume.py files in models/, schemas/ and routers/, so I filled those in. No other file changed.

- **models/resume.py:** each resume is tied to its user (user_id), and deleting a user deletes their resumes too. A resume stores the file's original name, the name it was saved under on disk (a random ID, so files never clash), its type, its size and the upload time.
- **schemas/resume.py:** this controls what the API sends back. The on-disk file name is left out, so users never see where files are stored.
- **routers/resume.py:** all three endpoints are under /api and require a logged-in user.
 - **POST /resume/upload** takes a real file in a field named file. It checks that the name ends in .pdf, that the browser says it's a PDF, and that the file actually starts the way every real PDF does. Empty files are rejected, and so are files over 5 MB, without loading the whole file into memory. It then saves the file and the database record. If saving the record fails, the file is deleted so nothing is left behind.
 - **GET /resume/mine** lists your own resumes, newest first.
 - **DELETE /resume/{id}** deletes one of your resumes, both the database record and the file. If the resume belongs to someone else, it answers "not found", as if the resume didn't exist, so users can't check whether other people's IDs are real.

Files are saved in backend/uploads/resumes/. You can change that with UPLOAD_DIR in .env. That folder gets its own .gitignore automatically, so resumes (personal data) can't be committed to the repo by accident.

Three things your team needs to know:

1. **Registration is broken on a fresh install, and it's not caused by these changes.** requirements.txt doesn't fix a version for bcrypt, so pip installs the newest one, which doesn't work with passlib 1.7.4. /api/register then crashes. Adding the line bcrypt==4.0.1 to requirements.txt fixes it. I didn't edit that file because it's shared, but whoever owns it should make the change.
2. **Anyone who already ran the old version needs to recreate the resume table.** The app only creates tables that don't exist yet; it never updates an existing one. Run DROP TABLE resumes; once in your local database, then restart the app.
3. **Uploads can't be done through the frontend's api.js helper as it is.** It always sends data as JSON, but a file upload has to be sent as a form (FormData), with the browser setting the content type itself. The upload call will need its own small function.

A natural next step would be a GET /resume/{id}/download endpoint, so users and recruiters can open the PDF.

[3 resume.py files attached]

AI-Suggested Content:

- A rewritten models/resume.py
- A rewritten schemas/resume.py
- A rewritten router/resume.py

Validation:

Compared the three AI files line by line with our versions and checked them against our existing User model (resumes relationship), database.py, routers/auth.py (get_current_user) and the models/ and schemas/ __init__.py imports. Tested upload, list and delete through the API docs at /docs with a real PDF.

Decision: Modified and generally accepted.

Reflection: The AI gave a clean, well-structured starting point that matched our models/schemas/routers layout. I simplified it to fit our sprint tasks, but some of the safety checks we dropped are worth adding back later.

Responsible Person: Diego Chidiac

3. Tasks 3.1: Set up file storage and define accepted formats and size limits (e.g. PDF, DOCX) AND Task 3.2: Implement resume API endpoints (upload, list, download, delete) linked to the user account

Purpose of AI Use: The dashboard couldn't load resumes and the browser reported CORS error.

Chat Link or Prompt/Response:

Prompt: My GET /resume/mine request is failing with what looks like a CORS error in the browser, but /profile works fine with the same setup. Here's the error that i get, help me figure out whether this is really a CORS issue or something else, and fix it.

Access to fetch at '<http://localhost:5000/api/resume/mine>' from origin '<http://localhost:5173>' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.

:5000/api/resume/mine:1 Failed to load resource: net::ERR_FAILED

dashboard:1 Access to fetch at '<http://localhost:5000/api/resume/mine>' from origin '<http://localhost:5173>' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.

:5000/api/resume/mine:1 Failed to load resource: net::ERR_FAILED

Response: This isn't really a CORS problem. The server crashes with a 500 error, and the browser shows that as a CORS error. I recreated your setup with an out-of-date resumes table and got exactly your symptoms: /profile returned 200, and /resume/mine returned a 500 with no Access-Control-Allow-Origin header.

What's happening:

1. Your local database still has the resumes table from the original placeholder version (columns filename, uploaded_at). The app only creates tables that don't exist yet; it never adds new columns to an existing one.
2. So /resume/mine asks for columns that aren't there, and Postgres fails with column resumes.original_filename does not exist.
3. FastAPI answers that crash from outside its CORS layer, so the 500 goes out with no CORS headers. The browser can't read a response without them, so it shows "blocked by CORS policy" instead of the real error. /profile works because nothing crashes there.

The real error was in the terminal running uvicorn, not in the browser. When a browser shows a CORS error on some endpoints but not others, check the server log first.

To fix it now, run this once in psql against your local database, then restart the server:

```
sql
DROP TABLE resumes CASCADE;
```

This deletes any test resumes you'd saved locally. On restart the table is recreated with the right columns. After that, a request from localhost:5173 returned 200 with correct CORS headers, and all the earlier resume tests still pass.

I also changed `app.py` so this can't hide the real error again:

- **Out-of-date tables are caught at startup.** If a table is missing columns, the server refuses to start and prints which table and columns are missing, with the command to run.
- **Server errors keep their CORS headers.** The browser now shows a normal 500 instead of a fake CORS error, and the full error details are logged in the terminal.
- **Your Vite port is allowed.** The project only allowed `localhost:3000`. I added `localhost:5173`, and you can change the list with `CORS_ORIGINS` in `.env`. The project as you sent it would have blocked `/profile` from 5173 too, so you or a teammate may have already changed this locally. If so, keep this version when merging.

`app.py` is a shared file, so let your team know about it. Everyone who ran the earlier version will hit the startup message once and need to run the same `DROP TABLE`.

AI-Suggested Content:

- **Diagnosis:** the error wasn't really a CORS problem. The server was crashing with a 500 error because the resumes table in the database was out of date.
- **Tip:** when a CORS error shows up on some endpoints but not others, check the uvicorn terminal for the real error first.
- **Suggested `app.py` changes**

Validation: Found the real error: a missing column. By comparing `models/resume.py` with the resume table in Supabase, I found that a column was missing (`filepath`). After fixing it, the error disappeared.

Decision: Modified and partially accepted.

Reflection: The AI helped me find the error by assuming a local database with some columns, so the exact fix didn't perfectly match but it did point me to the error. Now, I know that when a model changes, the Supabase table has to be updated by hand as well.

Responsible Person: Diego Chidiac

4. Tasks 3.1: Set up file storage and define accepted formats and size limits (e.g. PDF, DOCX) AND Task 3.2: Implement resume API endpoints (upload, list, download, delete) linked to the user account

Purpose of AI Use: Extending the resume upload feature to accept DOCX files and adding a download endpoint.

Chat Link or Prompt/Response:

Prompt: Extend the resume upload feature to accept DOCX files in addition to PDF, and add a download endpoint that returns the original file with the correct filename and content type. Keep the code commented well enough for someone else to follow.

Response: Uploads now accept DOCX as well as PDF, and there's a new GET `/api/resume/{id}/download` endpoint. I tested both against a real Postgres database, and the earlier PDF tests still pass. The database columns didn't change, so nobody needs to drop the table this time. Most of the work is in `routers/resume.py`, which now opens with a summary of the endpoints and the upload checks, and each part is commented.

DOCX uploads:

- **Content is checked, not just the file name.** Accepted and rejected formats are defined in one small table in the code, so adding another format later is one entry. The file name must end in `.pdf` or `.docx`, and the actual bytes must match that format. For DOCX, the file is opened as an archive and must contain the parts every Word document has, without unpacking anything.
- **Files that only look like Word documents are rejected:** a PDF renamed to `.docx`, an Excel file renamed to `.docx`, a random ZIP, and a Word file with macros renamed to `.docx`. Old `.doc` files are also rejected.
- **The upload type check is looser for DOCX.** On computers without Word installed, browsers often label a `.docx` upload as a generic type, so those labels are accepted for DOCX. The content check above is what protects you. The saved type is always the official DOCX type, whatever the browser said.

Download:

- **You get back the exact file you uploaded,** with the right content type and the original filename, including accented names like `Résumé - José.docx`. A downloaded DOCX opened correctly in a Word library.
- **Only the owner can download.** Another user's resume, a missing ID, or a deleted resume all give 404. If the database has the resume but the file is gone from disk, the user gets a clear 404 and the problem is logged on the server.

Two things for the frontend:

- **app.py has one new CORS line.** Browsers hide the header that carries the filename from JavaScript on a different port unless the server allows it, and now it does.
- **A plain link won't download.** An `<a href>` can't send the login token, so the page has to fetch the file with the token, then save it:

js

```
const res = await fetch(`${API_BASE}/resume/${id}/download`, { headers:
{ Authorization: `Bearer ${token}` } });
const name = decodeURIComponent(res.headers.get("Content-
Disposition").match(/filename\*=UTF-8'([^;]+)/)[1]);
const url = URL.createObjectURL(await res.blob());
Object.assign(document.createElement("a"), { href: url, download:
name }).click();
URL.revokeObjectURL(url);
```

[attached 2 resume.py files]

AI-Suggested Content:

- A rewritten models/resume.py
- A rewritten app.py
- A rewritten router/resume.py

Validation:

- Compared the AI's files line by line with our routers/resume.py, models/resume.py and app.py.
- Tested uploading a PDF and a DOCX, rejecting other file types, and downloading both through the dashboard. Downloaded files opened with their original names.
- The AI's code was built on its own version of the model (original_filename, stored_filename, content_type, file_size), not ours (filename, filepath), so it couldn't be dropped in as is.

Decision: Modified and partially accepted.

Reflection: The AI's design was well explained and its format table made DOCX support easy to add, but it was written for a different version of our model, so I had to rebuild it on our own columns and kept the simpler parts.

Responsible Person: Diego Chidiac